

Лабораторная работа № 4.

Отладка и логирование Node.js-приложений

Тема: Node.js Inspector, stack trace, Pino, request ID и диагностика ошибок Internet Shop

Цель: Научиться воспроизводить и локализовать ошибки Node.js/Express-приложения, использовать debugger и stack trace, внедрить structured logging с Pino, request ID и безопасную обработку ошибок без утечки секретов.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте проект express-shop. Должны работать Express Router, Controller/Service, Prisma/PostgreSQL, JWT, validation и integration tests. В этой работе вы намеренно создадите и диагностируете несколько ошибок.

1. Установите Pino

PowerShell

```
npm install pino
npm install --save-dev pino-pretty
```

2. Создайте logger

config/logger.js

```
import pino from "pino";

const isDevelopment =
  process.env.NODE_ENV !== "production";

export const logger = pino({
  level: process.env.LOG_LEVEL ?? "info",

  redact: [
    "password",
    "passwordHash",
    "accessToken",
    "refreshToken",
    "req.headers.authorization"
  ],

  ...(isDevelopment
    ? {
        transport: {
          target: "pino-pretty",
          options: {
            colorize: true
          }
        }
      }
    : {})
});
```

В development logs выводятся удобно для чтения. В production оставляйте структурированный JSON output.

3. Добавьте startup log

```
server.js

import { app } from "./app.js";
import { logger } from "../config/logger.js";

const PORT = Number(process.env.PORT ?? 3000);

logger.info({
  nodeEnv: process.env.NODE_ENV,
  port: PORT,
  databaseConfigured: Boolean(process.env.DATABASE_URL)
}, "Application starting");

app.listen(PORT, () => {
  logger.info({ port: PORT }, "Server started");
});
```

4. Создайте request ID middleware

```
middleware/requestContext.js

import { randomUUID } from "node:crypto";
import { logger } from "../config/logger.js";

export function requestContext(req, res, next) {
  const requestId =
    req.get("X-Request-Id") ?? randomUUID();

  req.requestId = requestId;
  req.log = logger.child({ requestId });

  res.set("X-Request-Id", requestId);

  next();
}
```

5. Подключите requestContext

```
app.js

import { requestContext } from "../middleware/requestContext.js";

app.use(requestContext);
```

6. Добавьте request logging middleware

```
middleware/requestLogger.js

export function requestLogger(req, res, next) {
  const startedAt = performance.now();
```

```

req.log.info({
  method: req.method,
  path: req.originalUrl
}, "Request started");

res.on("finish", () => {
  const durationMs =
    Math.round((performance.now() - startedAt) * 100) / 100;

  req.log.info({
    method: req.method,
    path: req.originalUrl,
    statusCode: res.statusCode,
    durationMs
  }, "Request completed");
});

next();
}

```

7. Подключите requestLogger

```

app.js

app.use(requestContext);
app.use(requestLogger);

```

8. Проверьте request ID

1. Запустите приложение.
2. Выполните GET /api/products.
3. Найдите requestId в server logs.
4. Проверьте header X-Request-Id в HTTP response.
5. Повторите request и убедитесь, что ID другой.

9. Создайте диагностируемую ошибку

Временно добавьте ошибку в Product Service: передайте в Prisma неверный тип id или намеренно используйте имя несуществующего свойства.

```

Временная ошибка для лабораторной

export async function findProductById(id) {
  return prisma.product.findUnique({
    where: {
      id: String(id)
    }
  });
}

```

После выполнения упражнения эту ошибку необходимо исправить.

10. Воспроизведите failure

6. Выполните GET `/api/products/1`.
7. Зафиксируйте HTTP status.
8. Прочитайте сообщение в terminal.
9. Найдите первую строку stack trace, относящуюся к вашему проекту.
10. Запишите файл и номер строки.

11. Запустите Node.js Inspector

PowerShell

```
node --inspect server.js
```

Подключитесь к процессу из IDE или Chrome DevTools.

12. Попробуйте --inspect-brk

PowerShell

```
node --inspect-brk server.js
```

Сравните поведение `--inspect` и `--inspect-brk`.

13. Поставьте breakpoint

11. Поставьте breakpoint в Product Controller.
12. Повторите GET `/api/products/1`.
13. Проверьте `req.params.id`.
14. Выполните Step Into в Service.
15. Проверьте переменную `id` перед Prisma query.
16. Найдите момент, когда число превращается в строку.

14. Используйте debugger statement

Временный вариант Controller

```
export async function getProduct(req, res, next) {
  try {
    const id = Number(req.params.id);

    debugger;

    const product = await findProductById(id);
    res.json(product);
  } catch (error) {
    next(error);
  }
}
```

После диагностики удалите `debugger;` из production code.

15. Исправьте ошибку

Исправленный Service

```
export async function findProductById(id) {
  return prisma.product.findUnique({
    where: { id }
  });
}
```

16. Добавьте безопасный error logging

middleware/errorHandler.js

```
export function errorHandler(error, req, res, next) {
  req.log?.error({
    err: error,
    method: req.method,
    path: req.originalUrl
  }, "Request failed");

  const status = error.status ?? 500;

  res.status(status).json({
    error: status < 500
      ? error.message
      : "Internal server error",
    requestId: req.requestId
  });
}
```

17. Проверьте, что stack trace не уходит клиенту

17. Снова временно вызовите unexpected error.
18. Проверьте server log: там должен быть Error и stack.
19. Проверьте HTTP response.
20. Убедитесь, что response содержит только безопасное сообщение и requestId.
21. Убедитесь, что filesystem paths и Prisma internals не возвращаются клиенту.

18. Проверьте redaction

Тестовый log

```
logger.info({
  email: "student@example.test",
  password: "secret123",
  accessToken: "eyJ.test.token"
}, "Redaction demo");
```

В output чувствительные значения должны быть скрыты. После проверки удалите demo log.

19. Добавьте domain context

Product Controller

```
req.log.info({
  productId: req.validated.id
}, "Loading product");
```

Не записывайте весь `req.body` без необходимости. Логируйте только минимально нужный диагностический context.

20. Диагностируйте 401

22. Вызовите protected endpoint без token.
23. Проверьте status 401.
24. Добавьте безопасный debug/info log в `requireAuth`.
25. Не логируйте сам Bearer token.
26. Логируйте `requestId` и причину отказа, например `missing_authorization`.

21. Диагностируйте ошибку PostgreSQL

Временно остановите PostgreSQL или укажите неправильный test/dev connection port. Выполните request, требующий Prisma query.

27. Зафиксируйте HTTP 500.
28. Найдите `requestId`.
29. По `requestId` найдите error log.
30. Определите, что проблема находится в DB connection, а не Router.
31. Верните правильную конфигурацию.

22. Добавьте regression test

После исправления ошибки `id` добавьте integration test, который подтверждает корректный `GET /api/products/:id`.

Пример проверки

```
const response = await request(app)
  .get(`/api/products/${product.id}`);

expect(response.status).toBe(200);
expect(response.body.id).toBe(product.id);
```

23. Самостоятельное задание

32. Создайте ошибку validation и сравните её с unexpected 500.
33. Добавьте `warn` log для 403 Forbidden.
34. Добавьте error log для unexpected 500.
35. Проверьте, что `requestId` одинаков во всех logs одного request.
36. Проверьте X-Request-Id в response.
37. Установите `LOG_LEVEL=debug` и сравните output.
38. Верните `LOG_LEVEL=info`.
39. Проверьте redaction password/token.
40. Убедитесь, что production-safe response не содержит stack.
41. Добавьте regression test минимум для одной исправленной ошибки.

Требования к результату

- Pino установлен.
- Создан общий logger.
- Настроены log levels.
- Настроен redact для чувствительных данных.
- Добавлен `requestId` middleware.

- Используется child logger.
- Request logs содержат method/path/status/duration.
- Error handler пишет stack в server log.
- Клиент не получает internal stack trace.
- Node Inspector используется для поиска причины.
- Студент умеет читать stack trace.
- Есть regression test после исправления.

Критерии оценивания

№	Критерий
1	Pino установлен и logger создан
2	Есть structured info log
3	Есть request ID
4	Request ID возвращается в response header
5	Используется child logger
6	Логируется HTTP status и duration
7	Passwords/tokens redacted
8	Unexpected error логируется с Error object
9	Client не получает stack trace
10	Студент использовал --inspect или --inspect-brk
11	Breakpoint применён для поиска ошибки
12	Студент объясняет stack trace
13	Ошибка DB connection распознана по logs
14	Исправленная ошибка подтверждена integration test
15	Студент объясняет различие debugging, logging и testing